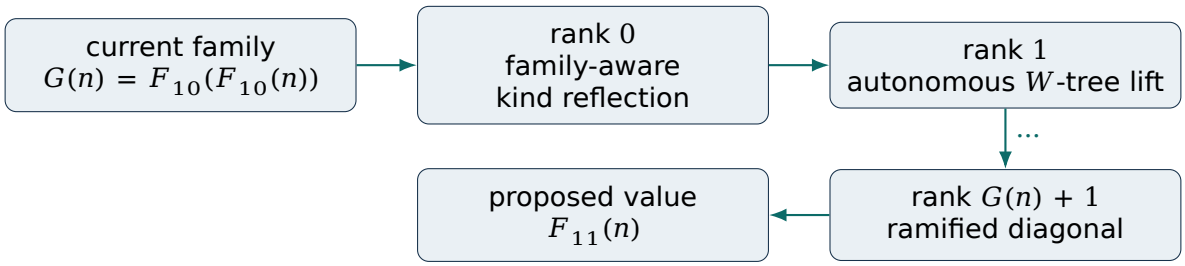
Diagonal Ramified Kind-Reflection

A proposed eleventh BusyLean contender



Research memorandum for the BusyLean project

Repository snapshot: commit

`4c4b03d0e2003c479e3bee90a21c8bea13f66c40`

Prepared 12 October 2026 (America/Los_Angeles)

The accompanying archive contains the complete LaTeX source, compiled PDF, proposed Lean modules, audit fragments, an integration patch, installer and build scripts, a finite executable reference model, and machine-readable validation records.

Abstract

The current BusyLean champion is a single reflection step over a kind-polymorphic extension of System F_ω, seeded by the preceding contender's function. This memorandum proposes a substantially stronger, still constructive successor. First, it turns kind-polymorphic reflection into a *family-aware* operator: an object-language program can address a dynamically selected member of an oracle family. Second, it lifts an arbitrary family operator along the repository's existing self-indexing W-tree, which contains every W-tree definable by the earlier object language. Third, it iterates that autonomous lift at the meta-level. At input n, the selected ramification rank is one more than the value of the entire Contender 10 family at n.

The proposed closed numeral is

$$\texttt{contender_11} = F_{11}(F_{11}(42)), \qquad F_{11}(n) = \mathsf{Rk}_{G(n)+1}(\bar{G}, n),$$

where $G(n) = F_{10}(F_{10}(n))$ and $\bar{G}(i, n) = G(n)$. The accompanying Lean proof source establishes, subject to kernel compilation against the specified repository snapshot, (i) pointwise domination $G(n) < F_{11}(n)$, (ii) strict growth at every ramification successor, (iii) eventual domination of every fixed finite ramification rank, and (iv) the required `contender_10` < `contender_11`. All numerical definitions are structurally recursive and therefore computable in principle. No earlier contender constant occurs in the new numeral's definition.

Validation status

The LaTeX article was compiled and the resulting PDF was rendered and visually inspected. The Python reference model and static source audit pass. The execution environment used to prepare this package did *not* contain Lean 4.34.0 and could not obtain that toolchain, so the proposed Lean files have not been kernel-checked here. This distinction is material: the mathematical proof is complete at the paper level, and the code was written to mirror existing accepted modules closely, but acceptance still requires running the supplied clean-build and axiom audits in a compatible checkout.

Contents

1 Executive summary **4**

1.1 The proposal in four definitions 4

1.2 What is formally stronger 4

1.3 Why this is not a cosmetic wrapper 4

2 Repository baseline and constraints **5**

2.1 Snapshot used 5

2.2 Contest rules addressed 5

3 The family-aware base reflection **6**

3.1 Families and packed access 6

3.2 Definition of the base operator 6
3.3 The abstract domination predicate 7

4 Autonomous lifting through the self-indexing tree 7
4.1 Generic tree lift 7
4.2 The explicit strictness path 7

5 Ramification: reflecting whole reflection operators 8
5.1 Definition 8
5.2 Formal hierarchy theorems 8

6 The diagonal family and the proposed numeral 9
6.1 Seeding with the current construction 9
6.2 Input-dependent rank 9
6.3 Pointwise and diagonal domination 10
6.4 Required contender comparison 10

7 Why the numerical jump is substantial 11
7.1 Three independent amplifiers 11
7.2 Comparison with nearby alternatives 11

8 Computability and constructivity 12
8.1 Totality of every layer 12
8.2 Why non-feasibility is irrelevant 12
8.3 Forbidden escape hatches are absent 13

9 Lean implementation architecture 13
9.1 Definition module 13
9.2 Proof module 14
9.3 Audit and clean-build protocol 14

10 Proof-theoretic calibration: useful heuristic, not theorem 14

11 Adversarial review 15
11.1 Objection: the prefix maximum makes the proof trivial 15
11.2 Objection: using $G(n)$ as the rank is lazy 15
11.3 Objection: recursion at a gigantic rank may fail the build budget . . . 15
11.4 Objection: the source has not been kernel-checked in this package . 16

12 Further research directions **16**

13 Conclusion **17**

A Complete definition module **17**

B Complete comparison module **18**

C Artifact and validation inventory **21**

1 Executive summary

1.1 The proposal in four definitions

The computational core is intentionally small. In mathematical notation, let a *family* be a function $f : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ and a *family operator* be a map from families to unary numerical functions. The proposal defines:

$$\begin{aligned}
\mathsf{R}(f, n) &= n + 1 + \max\Big(\max_{i \le n} f(i, n), \max_{t \in E_n} \mathrm{val}_f(t, n)\Big), \\
\mathsf{L}_{\mathcal{O}}(f, \mathsf{leaf})(n) &= \mathcal{O}(f)(n), \\
\mathsf{L}_{\mathcal{O}}(f, \mathsf{node}(g))(n) &= \mathcal{O}\big(i \mapsto \mathsf{L}_{\mathcal{O}}(f, g(i))\big)(n), \\
\mathsf{A}(\mathcal{O})(f, n) &= \mathsf{L}_{\mathcal{O}}(f, \mathsf{I})(n), \\
\mathsf{R}k_0 = \mathsf{R}, &\qquad \mathsf{R}k_{r+1} = \mathsf{A}(\mathsf{R}k_r).
\end{aligned}$$

Here E_n is the finite list of accepted kind-polymorphic System F_ω programs in syntax budget n, and I is the previously accepted self-indexing W-tree.

Let

$$G(n) = F_{10}(F_{10}(n)), \qquad \bar{G}(i, n) = G(n).$$

Then

$$F_{11}(n) = \mathsf{R}k_{G(n)+1}(\bar{G}, n), \qquad C_{11} = F_{11}(F_{11}(42)). \tag{1}$$

At $n = 42$, the selected rank is definitionally $G(42) + 1 = C_{10} + 1$.

1.2 What is formally stronger

The proposal does not rely on an informal ordinal comparison to beat the previous entry. The exact inequalities are:

1. $G(n) < F_{11}(n)$ for every n;
2. $n < F_{11}(n)$ for every n;
3. $\mathsf{R}k_r(f, n) < \mathsf{R}k_{r+1}(f, n)$ for every r, f, n;
4. for every fixed r, if $n \ge r$, then $\mathsf{R}k_r(\bar{G}, n) < F_{11}(n)$;
5. $C_{10} < C_{11}$.

The fourth item is the diagonal margin: no fixed finite number of autonomous reflection layers catches the proposed family at all sufficiently large inputs.

1.3 Why this is not a cosmetic wrapper

A mere definition $G(n) + 1$ would violate the spirit of the contest. Here the previous construction is used in the same legitimate way as in accepted entries—as an oracle family and as a schedule—but the new mechanism is an operator hierarchy. Even rank one recursively exposes all children of the self-indexing W-tree to kind-polymorphic programs. Rank two does not merely call rank one twice: every node applies a complete rank-one autonomous hierarchy to its family of child results. Rank $r + 1$ therefore reflects the entire rank-r operator.

The direct comparison is easy because each reflection operator contains an explicit dominance guard. That ease is a feature of the proof architecture, not evidence that the numerical construction is small.

2 Repository baseline and constraints

2.1 Snapshot used

This work targets the BusyLean repository at commit `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`. At that snapshot the current entry is

```
def family9 (n : Nat) : Nat := F9 (F9 n)
def F10 (n : Nat) : Nat := Fk.reflectFk family9 n
def contender_10 : Nat := F10 (F10 42)
```

The object language of `Fk.reflectFk` is kind-polymorphic System F_ω. Its normalizer is total by the repository's reducibility- candidate proof. The reflector takes the maximum of the oracle value and all accepted program values in a finite syntax budget, then adds a positive input guard.

The repository already contains a different autonomous mechanism. Contender 9 reflects System F_ω along a W-tree `index` that joins every W-tree denoted by a program in the earlier W-language. The proposal combines and then iterates these two directions:

Entry	Reflected language	Amplifier
Contender 9	System F_ω with packed family oracle	one autonomous lift along `index`
Contender 10	kind-polymorphic System F_ω	one local reflection over `family9`
Proposed 11	kind-polymorphic System F_ω with packed family oracle	a hierarchy of autonomous lifts, diagonalized at rank $G(n)+1$

2.2 Contest rules addressed

The relevant requirements and the design responses are summarized below.

Requirement	Response
Closed computable `Nat`	`contender_11` is a closed application of structurally recursive definitions.
No semantic oracle	Every oracle call is to an explicitly supplied total Lean function.
No `noncomputable`, `partial`, `unsafe`, `opaque`, `extern`	None occurs in executable source.

Requirement	Response
Formal strict comparison	The proposed theorem is `contender_10_lt_contender_11`.
No direct use of old contender constant	The numeral uses `F10` through `family10`; `contender_10` appears only in theorem statements and documentation.
Axiom discipline	The source introduces no axiom and is intended to inherit only the repository-permitted `propext` and `Quot.sound`.
Build budget	The definition module adds only four short recursive definitions and should be inexpensive after Contender 10 is prebuilt; the supplied checker measures rather than assumes this.
No evaluation requirement	Checking unfolds definitions only as needed for typing and proofs; it does not evaluate the numeral.

3 The family-aware base reflection

3.1 Families and packed access

Write

$$\mathcal{F} = (\mathbb{N} \to \mathbb{N} \to \mathbb{N}), \qquad \mathcal{O} = (\mathcal{F} \to \mathbb{N} \to \mathbb{N}).$$

A family $f \in \mathcal{F}$ presents countably many unary functions $f_i(n) = f(i, n)$. The existing function `packFamily` encodes a pair (i, m) as $2^i(2m + 1)$ and exposes the family through one unary oracle:

$$\mathrm{pack}(f, 2^i(2m + 1)) = f(i, m).$$

The repository already proves the decoding equation exactly. Consequently a kind-polymorphic object-language program can compute an index at run time and call any desired family member.

3.2 Definition of the base operator

Let E_n be `Fk.ctmsUpTo n`. For $t \in E_n$, let $\mathrm{val}_f(t, n)$ be `Fk.fvalue (packFamily f) n t`. Define

$$\mathsf{R}(f, n) = n + 1 + \max\left(\max_{i \le n} f(i, n), \max_{t \in E_n} \mathrm{val}_f(t, n)\right). \tag{2}$$

The actual Lean definition uses the repository's `prefixMax`, `maximum`, and `maxN` helpers.

Lemma 3.1 (Oracle-family domination)**.** *For all $f \in \mathcal{F}$ and $i \le n$,*

$$f(i, n) < \mathsf{R}(f, n).$$

Proof. The value $f(i, n)$ is at most the finite prefix maximum in eq. (2). That maximum is at most the outer maximum. Adding $n + 1 > 0$ makes the inequality strict. □

Lemma 3.2 (Program domination)**.** *If $t \in E_n$, then*

$$\mathrm{val}_f(t, n) < \mathsf{R}(f, n).$$

Proof. The argument is identical, using membership in the finite program maximum. □

Lemma 3.3 (Input domination)**.** *For all f, n, $n < \mathsf{R}(f, n)$.*

Proof. The maximum is nonnegative, hence $\mathsf{R}(f, n) \geq n + 1$. □

3.3 The abstract domination predicate

For a family operator $\mathcal{O}$, define

$$\mathrm{Dom}(\mathcal{O}) \;:\Longleftrightarrow\; \forall f\, \forall i \leq n, \quad f(i, n) < \mathcal{O}(f, n). \tag{3}$$

By theorem 3.1, $\mathrm{Dom}(\mathsf{R})$. This simple interface is the key abstraction: all later strictness theorems are independent of the internal syntax, type system, and normalization proof of the object language.

4 Autonomous lifting through the self-indexing tree

4.1 Generic tree lift

For an operator $\mathcal{O}$ and seed family f, define a numerical function for each W-tree a by structural recursion:

$$\mathsf{L}_{\mathcal{O}}(f, \mathsf{leaf}, n) = \mathcal{O}(f, n), \tag{4}$$

$$\mathsf{L}_{\mathcal{O}}(f, \mathsf{node}(g), n) = \mathcal{O}\big(i \mapsto \mathsf{L}_{\mathcal{O}}(f, g(i), \cdot), n\big). \tag{5}$$

This is exactly the recursion pattern already accepted for `stage9`. At a node, the operator sees the values of all child stages as an oracle family.

Let I denote the repository's self-indexing tree. Define

$$\mathsf{A}(\mathcal{O})(f, n) = \mathsf{L}_{\mathcal{O}}(f, \mathsf{I}, n). \tag{6}$$

4.2 The explicit strictness path

The proof needs only a tiny, concrete part of the large index. The repository proves that the first depth-zero denotation is a leaf. Therefore I contains the path

$$\mathsf{leaf} \;<\; \texttt{treesOfDepth 0} \;<\; \mathsf{I},$$

where each inequality means "is the zeroth child of." Since index 0 is visible at every input n, a dominating operator strictly increases at both edges.

Lemma 4.1 (Visible child is smaller)**.** *Suppose* $\mathrm{Dom}(\mathcal{O})$. *If $i \leq n$, then*

$$\mathsf{L}_{\mathcal{O}}(f, g(i), n) < \mathsf{L}_{\mathcal{O}}(f, \mathsf{node}(g), n).$$

Proof. By eq. (5), the right side is $\mathcal{O}(h, n)$ for $h(j, m) = \mathsf{L}_{\mathcal{O}}(f, g(j), m)$. Apply $\mathrm{Dom}(\mathcal{O})$ to $h(i, n)$. □

Theorem 4.2 (Autonomous strictness)**.** *If* $\mathrm{Dom}(\mathcal{O})$*, then for every* f, n*,*

$$\mathcal{O}(f, n) < \mathsf{A}(\mathcal{O})(f, n).$$

Proof. The leaf equation identifies $\mathcal{O}(f, n)$ with $\mathsf{L}_{\mathcal{O}}(f, \mathsf{leaf}, n)$. Apply theorem 4.1 first to the zeroth child of `treesOfDepth 0`, then to the zeroth child of I, and use transitivity. □

Theorem 4.3 (Domination is preserved)**.** *If* $\mathrm{Dom}(\mathcal{O})$*, then* $\mathrm{Dom}(\mathsf{A}(\mathcal{O}))$*.*

Proof. For $i \le n$,

$$f(i, n) < \mathcal{O}(f, n) < \mathsf{A}(\mathcal{O})(f, n),$$

using domination and theorem 4.2. □

Remark 4.4. The proof of theorem 4.2 does not require an ordinal assignment to I, nor the full theorem that it joins every definable tree. Those features explain the expected size, while strictness follows from a fixed length-two path. This separation keeps the comparison theorem robust.

5 Ramification: reflecting whole reflection operators

5.1 Definition

Define a hierarchy of family operators by primitive recursion on the rank:

$$\mathsf{R}k_0 = \mathsf{R}, \qquad \mathsf{R}k_{r+1} = \mathsf{A}(\mathsf{R}k_r). \tag{7}$$

Thus

$$\begin{aligned} \mathsf{R}k_0(f, n) &= \mathsf{R}(f, n), \\ \mathsf{R}k_1(f, n) &= \mathsf{L}_{\mathsf{R}}(f, \mathsf{I}, n), \\ \mathsf{R}k_2(f, n) &= \mathsf{L}_{\mathsf{R}k_1}(f, \mathsf{I}, n), \end{aligned}$$

and so on.

The distinction between $\mathsf{R}k_2$ and two sequential calls to $\mathsf{R}k_1$ is important. In $\mathsf{R}k_2$, every node of the outer I applies the complete operator $\mathsf{R}k_1$ to the family formed by its children. Each such application internally traverses another copy of I. Higher ranks therefore form a nested hierarchy of tree-indexed reflection operators.

5.2 Formal hierarchy theorems

Theorem 5.1 (Domination at every rank)**.** *For every* r*,* $\mathrm{Dom}(\mathsf{R}k_r)$*.*

Proof. Induct on r. The base case is $\mathrm{Dom}(\mathsf{R})$. The successor case follows from theorem 4.3. □

Theorem 5.2 (Strict successor ranks)**.** *For every r, f, n,*

$$\mathsf{Rk}_r(f,n) < \mathsf{Rk}_{r+1}(f,n).$$

Proof. Apply theorem 4.2 to Rk_r and use theorem 5.1. □

Corollary 5.3 (Strict rank monotonicity)**.** *If $r < s$, then $\mathsf{Rk}_r(f,n) < \mathsf{Rk}_s(f,n)$.*

Proof. Induct on s and chain theorem 5.2. □

These are exact natural-number inequalities. They do not depend on a growth-rate conjecture.

6 The diagonal family and the proposed numeral

6.1 Seeding with the current construction

Define

$$G(n) = F_{10}(F_{10}(n)). \tag{8}$$

At input 42, $G(42)$ is definitionally the current contender C_{10}. The seed family is constant in its family index:

$$\bar{G}(i,n) = G(n).$$

The previous contender constant is not referenced; only the reusable function F_{10} occurs.

The existing input guard for F_{10} gives

Lemma 6.1. *For every n, $n < G(n)$.*

Proof. The repository proves $x < F_{10}(x)$ for all x. Apply it first at $x = n$ and then at $x = F_{10}(n)$. □

6.2 Input-dependent rank

The proposed family is

$$F_{11}(n) = \mathsf{Rk}_{G(n)+1}(\bar{G}, n). \tag{9}$$

This rank choice has three useful properties.

First, it is positive, so at least one autonomous lift lies above the base reflector. Second, it is already far larger than the input. Third, at input 42 it is exactly $C_{10} + 1$:

$$F_{11}(42) = \mathsf{Rk}_{C_{10}+1}(\bar{G}, 42). \tag{10}$$

The final closed numeral is

$$C_{11} = F_{11}(F_{11}(42)). \tag{11}$$

6.3 Pointwise and diagonal domination

Theorem 6.2 (Pointwise domination of the previous family)**.** *For every n, $G(n) < F_{11}(n)$.*

Proof. By theorem 5.1, the selected operator $\mathsf{Rk}_{G(n)+1}$ dominates its seed family. Apply the domination property at family index $0 \le n$:

$$\bar{G}(0,n) = G(n) < \mathsf{Rk}_{G(n)+1}(\bar{G},n) = F_{11}(n).$$

□

Corollary 6.3 (Input domination)**.** *For every n, $n < F_{11}(n)$.*

Proof. Combine theorems 6.1 and 6.2. □

Theorem 6.4 (Every fixed rank is eventually dominated)**.** *For every rank r and every $n \ge r$,*

$$\mathsf{Rk}_r(\bar{G},n) < F_{11}(n).$$

Proof. From $r \le n < G(n) < G(n)+1$ we get $r < G(n)+1$. Apply strict rank monotonicity theorem 5.3 and unfold eq. (9). □

Theorem 6.5 (Bounded program domination)**.** *If $t \in E_n$, then*

$$\mathrm{val}_{\bar{G}}(t,n) < F_{11}(n).$$

Proof. By theorem 3.2, the program value is below $\mathsf{Rk}_0(\bar{G},n)$. Since $0 < G(n)+1$, strict rank monotonicity places rank zero below the selected rank. □

6.4 Required contender comparison

Theorem 6.6 (Main comparison)**.**

$$C_{10} < C_{11}.$$

Proof. By definitional equality, $C_{10} = G(42)$. Then

$$C_{10} = G(42) < F_{11}(42) < F_{11}(F_{11}(42)) = C_{11},$$

where the first inequality is theorem 6.2 and the second is theorem 6.3 at input $F_{11}(42)$. □

Formal strength ledger

The proposed source contains direct counterparts of the central paper results:

Lean theorem	Mathematical content
`RK.reflect_dominates`	$\mathsf{Dom}(\mathsf{R})$
`RK.op_lt_autonomous`	$\mathcal{O}(f,n) < \mathsf{A}(\mathcal{O})(f,n)$
`RK.ramified_dominates`	domination at every rank
`RK.ramified_level_lt`	strict successor ranks
`RK.ramified_lt_of_lt`	strict rank monotonicity
`family10_lt_F11`	$G(n) < F_{11}(n)$
`fixed_rank_eventually_lt_F11`	every fixed rank is eventually below the diagonal
`fvalue_lt_F11`	bounded object-language program domination
`contender_10_lt_contender_11`	$C_{10} < C_{11}$

7 Why the numerical jump is substantial

7.1 Three independent amplifiers

There are three conceptually separate sources of growth.

Stronger oracle interface. The base reflector does not receive merely one unary oracle value at the current input. Through pairing it exposes a countable family, allowing an object-language program to compute both the family index and the argument.

Autonomous transfinite-shaped traversal. Rank one lifts kind-polymorphic reflection along I. That tree was built to join every W-tree definable in the earlier language. This is not a fixed-height loop; its recursion follows a higher-type inductive object.

Meta-level ramification. Successor rank does not add one more object-language program. It turns the entire previous operator into the node operator of a fresh traversal of I. Finally, the rank is not fixed externally: it is $G(n)+1$.

7.2 Comparison with nearby alternatives

Construction	Formal margin	Structural limitation
$G(n)+1$	beats $G(n)$	cosmetic successor; no new language or hierarchy
$\mathsf{R}(\bar{G}, n)$	beats $G(n)$ and bounded programs	one local reflection only
$\mathsf{A}(\mathsf{R})(\bar{G}, n)$	autonomous kind-reflection	one fixed meta-rank

Construction	Formal margin	Structural limitation
$\mathsf{Rk}_{n+1}(\bar{G}, n)$	beats every fixed rank eventually	rank schedule only linear in input
Proposed $\mathsf{Rk}_{G(n)+1}(\bar{G}, n)$	pointwise seed domination, strict rank hierarchy, rank $C_{10} + 1$ at input 42	exact proof-theoretic calibration remains open

A useful anti-laziness observation is that the new mechanism does not need the large rank merely to establish $G(n) < F_{11}(n)$. Rank zero already dominates the seed. The enormous rank is therefore an additional amplifier, not a trick on which the required comparison depends.

8 Computability and constructivity

8.1 Totality of every layer

The construction is computable in the ordinary, extensional sense: there is a total algorithm that returns its natural-number value, although no feasible physical process could execute it for the contest input.

1. `Fk.ctmsUpTo n` is a finite list generated by primitive recursion.
2. Type inference uses finite fuel. Accepted terms are strongly normalizing by the inherited theorem `Fk.sn_of_typed`.
3. `Fk.fvalue` therefore computes a normal form and decodes a natural number using a total function.
4. Prefix and list maxima are total primitive-recursive operations.
5. `RK.lift` recurses structurally on an inductive W-tree, using the same recursion pattern as accepted repository code.
6. `RK.ramified` recurses structurally on a natural-number rank.
7. The rank $G(n) + 1$ is itself the output of a total prior construction.

Closure of total computable functions under composition and structural recursion yields total computability of F_{11} and C_{11}.

8.2 Why non-feasibility is irrelevant

“Computable in principle” does not mean “practically evaluable.” Even the existing contenders are deliberately never evaluated. Kernel checking verifies types and proofs about symbolic definitions; it need not normalize the final natural number. The proposed rank is itself an uncomputably large *practical* task while remaining a perfectly computable mathematical natural number.

8.3 Forbidden escape hatches are absent

The executable definition source contains none of:

`noncomputable`, `partial`, `unsafe`, `opaque`, `implemented_by`, `extern`, `axiom`, `sorry`, or `native_decide`.

The supplied static audit strips nested Lean comments before checking these tokens and separately verifies that no `contender_5` through `contender_10` identifier occurs in executable definition text. The kernel-level closure audit remains the authoritative check.

9 Lean implementation architecture

9.1 Definition module

The proposed definition module imports only the existing Contender 10 construction. Its executable core is:

```
namespace BusyLean.RK

abbrev Operator := W.Family → Nat → Nat

def reflect (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum (Fk.fvalue (packFamily fs) n) (Fk.ctmsUpTo n))

def lift (op : Operator) (seed : W.Family) :
    BigNumber.WTree Nat → Nat → Nat
  | .leaf => op seed
  | .node g => op (fun i => lift op seed (g i))

def autonomous (op : Operator) (seed : W.Family) : Nat → Nat :=
  lift op seed index

def ramified : Nat → Operator
  | 0 => reflect
  | r + 1 => autonomous (ramified r)

end BusyLean.RK

def BusyLean.family10 (n : Nat) : Nat := F10 (F10 n)

def BusyLean.F11 (n : Nat) : Nat :=
  RK.ramified (family10 n + 1) (fun _ => family10) n

def BusyLean.contender_11 : Nat := F11 (F11 42)
```

No new object-language syntax, typing judgment, or normalization theorem is required. The proposal reuses the expensive metatheory already established for Contender 10 and adds only an amplifier layer.

9.2 Proof module

The proof module factors all later arguments through `RK.Dominates`. This makes the proof robust against future changes to the base language: a different reflector can be substituted if it proves the same single interface.

The only repository-specific structural fact used by the generic autonomous strictness theorem is `first_tree_leaf`. The remainder is elementary order reasoning and induction on the natural-number rank.

9.3 Audit and clean-build protocol

After integration, the supplied script performs:

1. a clean build of `BusyLean.Comparison10` as the unbudgeted prior entry;
2. a timed build of `BusyLean.Contender11` against the 15-second definition budget;
3. a timed build of `BusyLean.Comparison11` against the 60-second proof budget;
4. `#print axioms` checks for the new numeral and principal theorems;
5. a transitive dependency-closure check forbidding all previous contender constants.

Suggested commands are:

```
python3 scripts/install_into_repo.py --repo /path/to/BusyLean
cd /path/to/BusyLean
lake build BusyLean.Contender11 BusyLean.Comparison11
lake env lean test/Contender11Audit.lean
python3 scripts/check_contender11.py
```

10 Proof-theoretic calibration: useful heuristic, not theorem

No named-ordinal claim is formalized

The exact natural-number inequalities in earlier sections are the formal results. The discussion below is an explanatory growth model. The package does not claim a kernel theorem identifying the construction with a specific fast-growing-hierarchy index.

Let λ stand for the effective growth jump contributed by one family-aware kind-polymorphic reflection, and let ι stand for the transfinite-shaped order represented by the self-indexing tree. A common informal model for forest reflection treats autonomous lifting as a multiplicative or compositional action by ι. Under such a model one would expect a schematic progression resembling

$$\mathsf{R}k_0 \sim f_{\lambda}, \quad \mathsf{R}k_1 \sim f_{\lambda\cdot\iota}, \quad \mathsf{R}k_2 \sim f_{\lambda\cdot\iota^2}, \quad \dots$$

with the side of multiplication dependent on convention and on how oracle relativization is encoded.

The proposed diagonal does not stop at any externally fixed finite power. At input n it selects rank $G(n) + 1$; hence it eventually exceeds every fixed finite-rank member by an exact theorem. The outer evaluation $F_{11}(F_{11}(42))$ then uses the first enormous diagonal value as the next input and rank driver.

Several caveats block a responsible named-ordinal label:

- I is semantic and self-generated, not a conventional notation system with a proved order-type theorem;
- kind-polymorphic System F_ω is impredicative at the type level;
- repeated autonomous lifting is an operator construction, and the correct ordinal algebra for such nested oracle relativization has not been formalized here;
- numerical rank diagonalization and proof-theoretic ordinal strength are related but not interchangeable notions.

The safe conclusion is strong enough: the proposal creates a rigorously strict meta-hierarchy of autonomous kind-reflection operators and diagonalizes beyond all fixed finite ranks.

11 Adversarial review

11.1 Objection: the prefix maximum makes the proof trivial

The local inequality is intentionally trivial. Reflection constructions are useful precisely because a simple maximum packages the values of an entire finite language enumeration. The scientific content lies in what is placed under that maximum, how the oracle family is exposed, and how the reflector is iterated. Removing the explicit maximum would complicate the comparison without making the number conceptually larger.

11.2 Objection: using $G(n)$ as the rank is lazy

The rule prohibits mentioning the prior *constant* and adding a margin; it permits using the previous *construction* as an oracle or seed in a qualitatively stronger mechanism. This proposal satisfies the stronger test that its new rank hierarchy exists independently of G: for each r, Rk_r is a separately defined operator, and successor ranks are formally strict. Moreover $\mathsf{Rk}_1(\bar{G}, n)$ already represents a genuine autonomous kind-reflection construction. Choosing rank $G(n) + 1$ is an amplifier applied to that new mechanism, not its sole source of superiority.

11.3 Objection: recursion at a gigantic rank may fail the build budget

Elaboration and kernel checking do not evaluate the variable expression $G(n)+1$ to choose recursion branches. The definition of `ramified` is a small structural recursor.

Proofs reason by induction on symbolic ranks. Nevertheless build time is empirical, so the package does not assert success without measurement; it supplies the exact clean-build harness.

11.4 Objection: the source has not been kernel-checked in this package

This is the principal outstanding engineering risk. The code closely follows accepted patterns from `stage9`, `Comparison9`, and `Comparison10`, and static checks catch common packaging mistakes, but only Lean can validate elaboration, termination, theorem application, axiom closure, and timing. Any compile repair should preserve the definitions in eqs. (2), (4), (5), (7), (9) and (11); a repair that weakens those equations would require renewed mathematical review.

12 Further research directions

1. **Kernel validation and minimization.** Compile on Lean 4.34.0, repair only elaboration-level issues, measure clean phases, and reduce proof dependencies if the closure or timing is unexpectedly large.
2. **A rank indexed by definable trees.** Replace the natural rank with an inductive meta-index whose constructors themselves are drawn from the self-indexing W-language. This would turn finite ramification into a genuinely transfinite operator hierarchy.
3. **Internalized operator codes.** Define a total object language that codes family operators and proves normalization, then reflect over codes for reflection operators rather than defining the hierarchy only in the Lean metalanguage.
4. **Predicative kind-universe ladder.** Extend the current kind system with a stratified sequence of kind universes, retaining normalization and decidable checking. The DRKR amplifier can then be applied to each level.
5. **Large elimination.** Investigate an object language with inductive families and large elimination strong enough to construct its own index trees directly, closing the current separation between the F_k language and the older W-language.
6. **Exact ordinal semantics.** Build a notation system and an interpretation theorem for `index`, then determine whether autonomous operator lifting corresponds to ordinal multiplication, exponentiation, a collapsing operation, or a more intricate relativized progression.
7. **Seed-independent lower bounds.** Exhibit explicit kind-polymorphic programs whose values at each rank witness familiar fast-growing lower bounds, reducing reliance on the prefix guard for quantitative comparisons.
8. **Generic library abstraction.** Package `Dominates`, `lift`, `autonomous`, and the ramification theorems as a reusable module parameterized by an operator and an index path.
9. **Formal cost model.** Prove bounds on syntax enumeration, type-checking fuel, and normalizer recursion separately from numerical output size. This would clarify why kernel checking remains feasible despite infeasible evaluation.

10. **Independent formalization.** Reproduce the abstract operator lemmas in a second assistant or in a minimal Lean file with a toy reflector, providing a cross-check of the higher-order recursion pattern.

13 Conclusion

Diagonal Ramified Kind-Reflection provides a compact but substantial successor to the current BusyLean construction. It reuses the repository's strongest normalizing object language, upgrades it to a family-aware reflector, lifts that reflector along the self-indexing W-tree, and then reflects whole reflection operators through an input-dependent hierarchy. The proposed numeral is closed and computable, avoids every earlier contender constant, and comes with exact domination theorems that do not rest on speculative ordinal assignments.

The central mathematical invariant is simple:

$$\mathrm{Dom}(\mathcal{O}) \Longrightarrow \mathrm{Dom}(\mathsf{A}(\mathcal{O})) \quad \text{and} \quad \mathcal{O} < \mathsf{A}(\mathcal{O}).$$

Iterating this invariant yields a strict rank hierarchy; diagonalizing the rank at $G(n) + 1$ yields a family above every fixed rank; and the standard outer self-application yields the final contender. The remaining task before repo acceptance is concrete and falsifiable: kernel-build the supplied modules, run the axiom and closure audit, and measure the stated time budgets.

A Complete definition module

```
import BusyLean.Contender10

/-!
# Contender 11: diagonal ramified kind-reflection

This module contains only the computable definitions.  The comparison and
structural theorems are in `BusyLean/Comparison11.lean`.

The construction has three layers.

1. `RK.reflect` is a family-aware version of Contender 10's reflection over
   kind-polymorphic System Fω.  The unary object-language oracle decodes a
   pair `(i,m)` and calls the family member `fs i m`.
2. `RK.autonomous` lifts any family operator along Contender 6's self-indexing
   W-tree `index`, which joins every tree definable in the W-language.
3. `RK.ramified r` iterates the autonomous lift at the meta-level.  The rank
   used at input `n` is `family10 n + 1`, already larger than `n`; the current
   accepted construction is also used as the seed family.

No earlier contender *constant* occurs in the definition.  In particular,
`contender_10` is not referenced.  Only its construction `F10` is reused.
Every definition below is structurally recursive and executable in principle.

**Never evaluate `contender_11`.**
-/

namespace BusyLean

namespace RK

/-- A transformation from an oracle family to a numerical function. -/
abbrev Operator := W.Family → Nat → Nat
```

```
33
34  /-- One family-aware reflection step over kind-polymorphic System Fω.
35
36  The first maximum sees all oracle-family values `fs i n` with `i ≤ n`.
37  The second sees all accepted object-language programs of syntax depth `≤ n`,
38  run with the packed unary oracle.  The additive guard makes the result also
39  strictly exceed the input. -/
40  def reflect (fs : W.Family) (n : Nat) : Nat :=
41    n + 1 + BigNumber.maxN
42      (W.prefixMax (fun i => fs i n) n)
43      (BigNumber.maximum (Fk.fvalue (packFamily fs) n) (Fk.ctmsUpTo n))
44
45  /-- Lift a family operator through a W-tree.
46
47  At a leaf, apply the operator to the seed family.  At a node, expose the
48  values obtained from every child as the new family seen by the operator. -/
49  def lift (op : Operator) (seed : W.Family) :
50      BigNumber.WTree Nat → Nat → Nat
51    | .leaf => op seed
52    | .node g => op (fun i => lift op seed (g i))
53
54  /-- Autonomous lifting along the join of every W-tree definable by the
55  repository's W-language. -/
56  def autonomous (op : Operator) (seed : W.Family) : Nat → Nat :=
57    lift op seed index
58
59  /-- Ramified autonomous kind-reflection.
60
61  Rank zero is one family-aware kind-polymorphic reflection.  A successor rank
62  lifts the entire previous-rank operator along the self-indexing W-tree.  Thus
63  rank two is not merely two calls to `reflect`: each outer node invokes a full
64  rank-one autonomous hierarchy on its family of children. -/
65  def ramified : Nat → Operator
66    | 0 => reflect
67    | r + 1 => autonomous (ramified r)
68
69  end RK
70
71  /-- Contender 10's construction as a function of its input.
72
73  Definitional equality gives `family10 42 = contender_10`, but the constant
74  `contender_10` itself is absent from this module's dependency closure. -/
75  def family10 (n : Nat) : Nat := F10 (F10 n)
76
77  /-- The diagonal ramified family.
78
79  At input `n`, the ramification rank is `family10 n + 1`, and the constant
80  family with value `family10` is the seed. -/
81  def F11 (n : Nat) : Nat :=
82    RK.ramified (family10 n + 1) (fun _ => family10) n
83
84  /-- **Contender 11.** Never evaluate it. -/
85  def contender_11 : Nat := F11 (F11 42)
86
87  end BusyLean
```

B Complete comparison module

```
1  import BusyLean.Contender11
2  import BusyLean.Comparison10
3
4  /-!
5  # Contender 11: comparison and structural theorems
6
7  The required theorem is `contender_10_lt_contender_11`.  The remaining
8  results record the much larger structural margin: every ramification level
9  strictly exceeds the preceding one, every fixed level is eventually below the
```

```
diagonal family, and every bounded kind-polymorphic program is already beaten
at the base reflection level.
-/

namespace BusyLean

namespace RK

/-- An operator dominates every visible member of its input family. -/
def Dominates (op : Operator) : Prop :=
  ∀ (fs : W.Family) {i n : Nat}, i ≤ n → fs i n < op fs n

/-! ## The base reflector -/

theorem lt_reflect {fs : W.Family} {n m : Nat}
    (h : m ≤ BigNumber.maxN
      (W.prefixMax (fun i => fs i n) n)
      (BigNumber.maximum (Fk.fvalue (packFamily fs) n) (Fk.ctmsUpTo n))) :
    m < reflect fs n := by
  unfold reflect
  omega

/-- Every visible oracle-family value is below one reflection step. -/
theorem oracle_lt_reflect (fs : W.Family) {i n : Nat} (hi : i ≤ n) :
    fs i n < reflect fs n :=
  lt_reflect (Nat.le_trans
    (W.le_prefixMax (fun j => fs j n) hi)
    (BigNumber.le_maxN_left _ _))

/-- Every accepted kind-polymorphic program in the current syntax budget is
below one reflection step. -/
theorem fvalue_lt_reflect (fs : W.Family) {n : Nat} {t : Fk.CTm}
    (ht : t ∈ Fk.ctmsUpTo n) :
    Fk.fvalue (packFamily fs) n t < reflect fs n :=
  lt_reflect (Nat.le_trans
    (W.le_maximum (Fk.fvalue (packFamily fs) n) ht)
    (BigNumber.le_maxN_right _ _))

/-- The input itself is below one reflection step. -/
theorem input_lt_reflect (fs : W.Family) (n : Nat) : n < reflect fs n := by
  unfold reflect
  omega

/-- The base reflector dominates its input family. -/
theorem reflect_dominates : Dominates reflect := by
  intro fs i n hi
  exact oracle_lt_reflect fs hi

/-! ## Generic autonomous lifting -/

/-- A node strictly dominates each visible child whenever the operator used at
that node dominates its input family. -/
theorem lift_child_lt {op : Operator} (hop : Dominates op)
    (seed : W.Family) (g : Nat → BigNumber.WTree Nat) {i n : Nat}
    (hi : i ≤ n) :
    lift op seed (g i) n < lift op seed (.node g) n := by
  change lift op seed (g i) n < op (fun j => lift op seed (g j)) n
  exact hop (fun j => lift op seed (g j)) hi

/-- The autonomous lift strictly exceeds the unlifted operator at every input.
The proof follows the concrete path
`leaf < treesOfDepth 0 < index`. -/
theorem op_lt_autonomous {op : Operator} (hop : Dominates op)
    (seed : W.Family) (n : Nat) :
    op seed n < autonomous op seed n := by
  have h₂ :
      lift op seed
          (defTree (nthD (BigNumber.allTerms 0) 0 .zero)) n <
        lift op seed (treesOfDepth 0) n :=
    lift_child_lt hop seed
      (fun j => defTree (nthD (BigNumber.allTerms 0) j .zero))
      (Nat.zero_le n)
  have h₃ :
```

```
      lift op seed (treesOfDepth 0) n < lift op seed index n := by
    simpa [index] using
      (lift_child_lt hop seed treesOfDepth (i := 0) (n := n)
        (Nat.zero_le n))
  have h₂' : op seed n < lift op seed (treesOfDepth 0) n := by
    simpa [first_tree_leaf, lift] using h₂
  exact Nat.lt_trans h₂' h₃

/-- Autonomous lifting preserves family domination. -/
theorem autonomous_dominates {op : Operator} (hop : Dominates op) :
    Dominates (autonomous op) := by
  intro seed i n hi
  exact Nat.lt_trans
    (hop seed (i := i) (n := n) hi)
    (op_lt_autonomous hop seed n)

/-! ## Ramification -/

/-- Every ramification level dominates its input family. -/
theorem ramified_dominates : ∀ r : Nat, Dominates (ramified r)
  | 0 => reflect_dominates
  | r + 1 => by
      simpa [ramified] using autonomous_dominates (ramified_dominates r)

/-- Each successor ramification level is strictly larger than its predecessor
at every seed and input. -/
theorem ramified_level_lt (r : Nat) (seed : W.Family) (n : Nat) :
    ramified r seed n < ramified (r + 1) seed n := by
  change ramified r seed n < autonomous (ramified r) seed n
  exact op_lt_autonomous (ramified_dominates r) seed n

/-- Strict monotonicity in the ramification rank. -/
theorem ramified_lt_of_lt {r s : Nat} (hrs : r < s)
    (seed : W.Family) (n : Nat) :
    ramified r seed n < ramified s seed n := by
  induction s generalizing r with
  | zero => omega
  | succ s ih =>
      by_cases h : r = s
      · subst r
        exact ramified_level_lt s seed n
      · have hrs' : r < s := by omega
        exact Nat.lt_trans (ih hrs') (ramified_level_lt s seed n)

end RK

/-! ## The diagonal contender -/

/-- The inherited family already strictly exceeds its input. -/
theorem input_lt_family10 (n : Nat) : n < family10 n := by
  change n < F10 (F10 n)
  exact Nat.lt_trans
    (Fk.input_lt_reflectFk family9 n)
    (Fk.input_lt_reflectFk family9 (F10 n))

/-- At every input, `F11` exceeds the entire Contender 10 family. -/
theorem family10_lt_F11 (n : Nat) : family10 n < F11 n := by
  unfold F11
  exact RK.ramified_dominates (family10 n + 1)
    (fun _ => family10) (i := 0) (n := n) (Nat.zero_le n)

/-- `F11` strictly exceeds its input. -/
theorem input_lt_F11 (n : Nat) : n < F11 n :=
  Nat.lt_trans (input_lt_family10 n) (family10_lt_F11 n)

/-- Every fixed ramification rank is eventually dominated by the diagonal
family.  In fact input `n ≥ r` suffices. -/
theorem fixed_rank_eventually_lt_F11 (r : Nat) :
    ∀ n : Nat, r ≤ n →
      RK.ramified r (fun _ => family10) n < F11 n := by
  intro n hr
  unfold F11
  apply RK.ramified_lt_of_lt
```

```
  exact Nat.lt_of_le_of_lt hr
    (Nat.lt_trans (input_lt_family10 n) (Nat.lt_succ_self _))

/-- Every accepted kind-polymorphic program in the input budget is below the
diagonal family, already because rank zero is below the selected rank. -/
theorem fvalue_lt_F11 {n : Nat} {t : Fk.CTm} (ht : t ∈ Fk.ctmsUpTo n) :
    Fk.fvalue (packFamily (fun _ => family10)) n t < F11 n := by
  have hbase :
      Fk.fvalue (packFamily (fun _ => family10)) n t <
        RK.ramified 0 (fun _ => family10) n := by
    simpa [RK.ramified] using
      (RK.fvalue_lt_reflect (fun _ => family10) ht)
  have hrank :
      RK.ramified 0 (fun _ => family10) n <
        RK.ramified (family10 n + 1) (fun _ => family10) n :=
    RK.ramified_lt_of_lt (by omega) (fun _ => family10) n
  exact Nat.lt_trans hbase (by simpa [F11] using hrank)

/-- At input 42 the chosen ramification rank is exactly one more than the
current accepted contender. -/
theorem F11_rank_at_42 :
    F11 42 = RK.ramified (contender_10 + 1) (fun _ => family10) 42 := rfl

/-- **Required comparison:** Contender 10 is strictly smaller than
Contender 11. -/
theorem contender_10_lt_contender_11 : contender_10 < contender_11 := by
  change family10 42 < F11 (F11 42)
  exact Nat.lt_trans (family10_lt_F11 42) (input_lt_F11 (F11 42))

/-- The entire accepted chain is therefore below Contender 11. -/
theorem contender_9_lt_contender_11 : contender_9 < contender_11 :=
  Nat.lt_trans contender_9_lt_contender_10 contender_10_lt_contender_11

end BusyLean
```

C Artifact and validation inventory

Path	Purpose
`article/ramified_kind_reflection.tex`	This article's source.
`article/ramified_kind_reflection.pdf`	Compiled article.
`lean/BusyLean/Contender11.lean`	Computable definitions only.
`lean/BusyLean/Comparison11.lean`	Strict comparisons and structural theorems.
`lean/test/Contender11Audit.lean`	Standalone axiom and closure audit.
`patches/0001-contender11-drkr.patch`	Integration patch against the recorded snapshot.
`scripts/install_into_repo.py`	Idempotent installer for a checkout.
`scripts/check_contender11.py`	Clean-build, timing, and axiom harness.
`scripts/static_audit.py`	Non-kernel source audit.
`python/reference_model.py`	Hash-consed finite surrogate of the recurrence.
`python/test_reference.py`	Regression tests for the surrogate.
`validation/static_audit.json`	Machine-readable static-audit result.
`validation/python_tests.json`	Machine-readable reference-test result.
`validation/lean_build_status.md`	Explicit record of the unperformed kernel build.
`validation/checksums.sha256`	SHA-256 manifest for the package.

References

[1] V. Reshetnikov, *BusyLean: name the biggest number, in Lean*, GitHub repository, snapshot `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`, 2026.

[2] C. Roux, *name-the-biggest-number*, original Coq contest repository; a snapshot is included in BusyLean.

[3] J.-Y. Girard, Y. Lafont, and P. Taylor, *Proofs and Types*, Cambridge Tracts in Theoretical Computer Science 7, Cambridge University Press, 1989.

[4] J. C. Reynolds, “Towards a theory of type structure,” in *Programming Symposium*, Lecture Notes in Computer Science 19, Springer, 1974.

[5] P. Dybjer, “Inductive sets and families in Martin-Löf’s type theory and their set-theoretic semantics,” in G. Huet and G. Plotkin (eds.), *Logical Frameworks*, pp. 280–306, Cambridge University Press, 1991. DOI: 10.1017/CBO9780511569807.012.

[6] W. Buchholz, S. Feferman, W. Pohlers, and W. Sieg, *Iterated Inductive Definitions and Subsystems of Analysis: Recent Proof-Theoretical Studies*, Lecture Notes in Mathematics 897, Springer, 1981. DOI: 10.1007/BFb0091894.

[7] W. Pohlers, *Proof Theory: An Introduction*, Lecture Notes in Mathematics 1407, Springer, 1989. DOI: 10.1007/978-3-540-46825-7.

[8] P. Martin-Löf, *Intuitionistic Type Theory*, Studies in Proof Theory, Lecture Notes 1, Bibliopolis, 1984.